mailers. In Rails, a mailer is a class that's stored in the `app/mailers` directory. It contains one or more methods, with each method corresponding to an email template. To create the body of the email, these methods in turn use views (in the same way that controller actions use views to create HTML and XML). So let's create a mailer for our store application. We'll use it to send two different types of email: one when an order is placed and a second when the order ships. The `rails generate mailer` command takes the name of the mailer class along with the names of the email action methods:

```
depot> bin/rails generate mailer Order received shipped
    create  app/mailers/order_mailer.rb
    invoke  tailwindcss
    create    app/views/order_mailer
    create    app/views/order_mailer/received.text.erb
    create    app/views/order_mailer/received.html.erb
    create    app/views/order_mailer/shipped.text.erb
    create    app/views/order_mailer/shipped.html.erb
    invoke  test_unit
    create    test/mailers/order_mailer_test.rb
    create    test/mailers/previews/order_mailer_preview.rb
```

Notice that we create an `OrderMailer` class in `app/mailers` and two template files, one for each email type, in `app/views/order`. (We also create a test file; we'll look into this in *Testing Email*.)

Each method in the mailer class is responsible for setting up the environment for sending an email. Let's look at an example before going into detail. Here's the code that was generated for our `OrderMailer` class, with one default changed:

rails7/depot_q/app/mailers/order_mailer.rb

```
  class OrderMailer < ApplicationMailer
»   default from: 'Sam Ruby <depot@example.com>'

    # Subject can be set in your I18n file at config/locales/en.yml
    # with the following lookup:
    #
    #   en.order_mailer.received.subject
    #
    def received
```